

1. Introduction

‘Where shall I begin, please your Majesty?’ he asked. ‘Begin at the beginning,’ the King said, very gravely, ‘and go on till you come to the end; then stop.’

Lewis Carroll, *Alice’s Adventures in Wonderland*

During late 1999, the world waited, with growing concern, for the change of the calendar into the year 2000. The focus was on some potential damage from online computers that control vital systems. This damage could occur because of the mere change of the calendar year and the way years of the twentieth century were traditionally represented in computer memory, using only the least two significant digits from 00 to 99. This surprisingly small detail made some people expect extreme damage. It could have affected electronic systems driven by software, such as verifying traffic control, atomic missiles, nuclear reactors, banking systems, pension plans, electricity and water supply. The US alone spent over 100 billion dollars on combating this, so called, ‘Y2K-bug.’ Just prior to that date change, some people had escaped into self made shelters, while flashlights and bottled water were a popular demand. Joint teams of the US and Russian military spent the night of December 31 1999 at the North American Aerospace Defense Command (NORAD). Together they monitored the world’s skies, as a precaution against a possible computer error that could cause an unruly launch of missiles. Midnight, December 31 1999 has passed into the new millenium with no significant events, except for a few minor glitches.

Computer systems control many aspects of our lives. Telephone systems, store checkout registers, ticket reservation systems, medical systems, financial systems, are all highly computerized. Data communication between computers replaces, in most cases, the use of actual paper money transfer. Computers are even responsible for many of the activities required for flying commercial airplanes. Failure of computerized systems have already caused grave consequences, including fatal accidents, shutting down of vital systems, and loss of money.

The software development industry has grown over the last few decades at an unprecedented pace. Hardware, and in particular memory costs, kept decreasing. The internet has practically transformed the world into a big

computer network, where communication, information and business are constantly taking place on-line. With these technological changes, the software development industry has also transformed. It is no longer feasible for a talented individual to single-handedly develop a whole software system. Instead, tens or even thousands of programmers are taking part in joint development efforts. Different parts of programs are being developed by programmers who may be working in different locations, and perhaps have never seen each other, producing thousands or even millions of lines of code.

It is thus not surprising that controlling the quality of the software production process has become significantly difficult. Assuring that the products of different software development groups will successfully work together is a nontrivial task. One of the main techniques to combat this problem is the application of appropriate design methodologies. Software engineering suggests various techniques for increasing the quality of software products.

Even when coping with many rules of good design and programming, as dictated by various software engineering methods, programs may still contain errors. Different statistics show that a few errors are highly likely to occur per every thousand lines of code, even in code developed by very experienced and highly skilled programmers. It is thus very important to use methods that attempt to remove man-made mistakes from program code. This is the goal of *formal methods techniques*. While various software engineering methods try to guide the software development, formal methods attempt to accompany the development with techniques and tools for finding and point out potential problems.

1.1 Formal Methods

Formal methods are a collection of notations and techniques for describing and analyzing systems. These methods are *formal* in the sense that they are based on some mathematical theories, such as logic, automata or graph theory. They are aimed at enhancing the quality of systems. Formal specification techniques introduce a precise and unambiguous description of the properties of systems. This is useful in eliminating misunderstandings, and can be used further for debugging systems. Formal analysis techniques can be used to verify that a system satisfies its specification, or to systematically seek for cases where it fails to do so. Formal methods can reduce the time it takes to find design or coding errors. They also significantly reduce the risk of damage incurred by not finding an error in a system before deploying it. In this book we will focus on formal methods for software, which we will also refer to as *software reliability methods*.

These techniques are sometimes used by the programmers themselves, but often are the responsibility of special software quality assurance teams. In the latter case, the separation and cooperation between the developers and the people taking care of reliability (e.g., testing teams) creates a beneficial

synergy towards the development of more reliable software. In many cases, the quality assurance team is, in fact, bigger than the development team.

In the early years of formal methods research, the focus was on assuring the *correctness* of systems. Correctness of a system as used here means that the system satisfies its provided specification. *Deductive software verification* was one of the main formal methods techniques studied. Pioneers in the field, such as Dijkstra, Floyd, Gries, Hoare, Lamport, Manna, Owicki, Pnueli and others, developed different proof systems that could be applied for verifying the correctness of programs. The goal of such techniques was to formally verify critical systems, e.g., missiles, air traffic control. It was also suggested that verification methods could be used to assist with the development of software and hardware systems [94]. According to this suggestion, the programs were supposed to be refined in a stepwise manner, starting from their formal specification, and ending with actual code, in a way that would preserve the correctness of each refinement step.

Some limitations of formal methods were soon realized. Software verification methods do not guarantee the correctness of *actual* code, but rather allow one to verify some abstract *model* of it. Hence, due to possible differences between the actual system and its abstract model, the correctness of the proof may not carry over. In addition, the correctness proof can itself be incorrect. The verification process often captures only a small part of the functionality of the system. The reason is that the verification is done with respect to a given specification, which is formed manually, and is sometimes incomplete, omitting some important properties of the system. In fact, it is often very hard or even impossible to assess when a given specification is complete. Although some parts of deductive verification can be automated, it is, in principle, a manual technique. As such, deductive verification was demonstrated mainly for rather small examples and requires a lot of time and expertise.

Despite its limitations, deductive software verification has had several important successes. It has affected the software development process, by defining the notion of an *invariant*. An invariant is a correctness assertion that needs to hold at certain control points during execution. It asserts some connections between the program variables. When programmers add invariants to the code of a program, they gain more intuition about the way it works. Moreover, they can insert simple run-time checks that the invariant assertions really hold at the appropriate control points of the program. If an invariant fails to hold, the normal execution of the program is interrupted with a warning message.

Recently, new and powerful tools are being developed for supporting deductive software verification. These tools attempt to mechanize parts of the proofs. The cleanroom method [98, 99] (which will be described later in this book) is an example of an informal use of software verification. It is used

within a methodology, together with other software reliability methods, and aims at providing high quality software.

Early automatic verification techniques were suggested by West and Zafiropulo [146, 150]. These methods could be applied to various *finite state systems*, such as hardware circuits and communication protocols. *Model checking*, i.e., verifying a finite state system against a given temporal specification, was pioneered by Clarke and Emerson [28, 42] in the US, and Quielle and Sifakis [120] in France. Starting with modest examples such as traffic light controllers, these methods have matured into a multimillion dollar industry of tools.

New automatic and semi-automatic verification techniques, such as binary decision diagrams [24], partial order reduction [55, 113, 142], symmetry reduction [43], induction [81, 149], abstraction and others, demonstrated the scalability of automatic verification methods. These techniques gave evidence that verification can work well in practice, and not only in theory. However, several limitations still need to be taken into account when using automatic verification. These include severe inefficiency in handling nontrivial data structures, e.g., queues or trees. Furthermore, automatic verification methods are usually restricted to finite state systems. Thus, they are most appropriate for verifying communication protocols or some abstract representation of algorithms. They may fail to handle full fledged programs with *real* and *integer* variables and with pointer and array references. Like deductive verification, the automatic verification techniques also require modeling the system rather than verifying it directly. This may again create a discrepancy between the verified object and the actual system.

Software testing is perhaps the most frequently used quality assurance method. Instead of trying to provide a comprehensive check of a system, testing is focused on sampling the executions, according to some coverage criteria, and comparing the actual behavior with the behavior that is expected according to the specification. This is often done based on experimental evidence and experience, and is frequently performed in a rather informal way. Testing methods have the advantage of being able to check an actual system, rather than a model of it. Unlike model checking, testing is not limited to finite state systems. However, testing is not as comprehensive a technique as deductive verification or model checking. It does not cover all the possible executions. Thus, even code that has been thoroughly tested can still contain errors.

Some Formal Methods Terminology

The term *verification* is used in different ways by researchers and practitioners of diverse formal methods. In some cases, verification is used to refer only to the process of obtaining the formal correctness proof of a system, thus suggesting the narrow meaning of using deductive verification. In other con-

texts, verification is used to describe any action that is taken in an attempt to find errors in a program, including model checking and testing.

Since this book describes different formal methods techniques, we will sometimes have to choose interpretations of terms that may not be consistent across all the different communities. By means of *verification*, we will refer to the process of applying a manual or an automatic technique that is supposed to establish whether the code either satisfies a given property or behaves in accordance with some higher-level description of it. By this we will mostly refer to the activities of deductive verification and model checking, but not to testing, since the latter is more akin to *sampling* than to an exhaustive correctness checking.

We will distinguish between verifying that a software satisfies certain specification *properties* (this is sometimes called *validation* [70]), and verifying the *conformance* between two descriptions of the software. In the latter case, one description is often less detailed, i.e., more abstract.

1.2 Developing and Acquiring Formal Methods

Formal methods include specification, verification and testing techniques. These methods are used to enhance the quality of software development and hardware design. In the past two decades, the benefits, advantages, trade-offs and limitations of using different methods were realized. Understanding the limitations of formal methods is not less important than presenting their success stories. For example, experience shows that algorithmic methods are usually preferred over methods that require considerable human skills. On the other hand, automatic methods are limited in scope, e.g., are often restricted to finite state systems of moderate size. As a manifestation of the maturity of formal methods research, methods that attempt to *guarantee* correctness are treated with suspicion, while methods that are aimed at finding errors are preferred.

Tools and support for various formal methods were first pioneered by researchers in universities and in research laboratories. In recent years, we have witnessed companies, especially in the communication and in the hardware industries, where reliability is critical, developing their own formal methods and tools. In other cases, an interface is developed for translating the software into a form that is acceptable by some existing tool. The output of that tool may be translated back into some required format. We are also starting to see available tools that can enhance software reliability. However, the majority of these tools are related to various tasks of software testing or software modeling.

The research in formal methods is relatively new. Selecting the right technique and tool is difficult. Developing a new verification or testing tool requires considerable effort. In this book, we will try to point out the similarities

and differences between the various formal methods, and how they can be combined and complement each other.

When a new software product is developed, a decision must be made on whether to use a particular software reliability tool or technique. In some cases, an attempt should be made to develop a new tool or tailor an existing one to some specific needs. Different considerations may be taken into account by people in different roles in software development organizations. Some of the relevant issues are listed below.

- **Project manager**

- What are the benefits of using formal methods in my software project?
- What is the investment in time, human resources, money?
- Should I start an in-house formal methods group to develop new tools that will match my organization needs?

- **Head of quality assurance team**

- In what stages of the development should we use formal methods?
- How do I schedule the use of formal methods with the other development activities?
- How much human resources do we need for this task? What are their qualifications?
- How do we hire or train people to apply formal methods?

- **Engineer (User)**

- What are the best tools or techniques for achieving better reliability?
- How much would it cost?
- What is the support that is provided with the tools?
- What is the most appropriate formal representation of the system that is being developed?
- What is the probability of finding errors by using the different formal methods?
- Which specification formalism will best suit the developed system?
- How much time would it take to develop new methods or improve and fit existing ones to my organization's purposes?
- Do we need special equipment or software to support such tools?
- How do we model the developed system or a part of it using the particular method or tool selected?
- How can we interpret the results, i.e., is there really an error when the tool reports one, and how assured are we when no errors are found?
- What do we do when the tool we use fails, e.g., when the verification does not terminate in a reasonable time or when it runs out of memory?
- Are our specifications correct? Do they include internal contradictions? Do they include all the necessary requirements from the system?

- **Formal methods researcher**

- How can I increase the *expressiveness* of formal methods techniques, allowing the specification and verification of more system requirements?

- How can I increase the *efficiency* of the techniques, making the methods and tools faster and capable of handling larger instances?
- Are there good *heuristics* that work better than standard methods in many practical cases?
- How do I transfer new technology to potential users?
- What is the common notation used by software developers? How do I integrate it into the techniques and tools I develop?

We cannot answer in this book all of these questions, as some of them are strongly dependent on the particular system being developed, the environment and the organization. However, we will attempt to address these issues by providing information about some of the modern formal methods, their uses and limitations. Our goal is to describe the main methods, and present their strengths and weaknesses. This will allow potential users to acquire more knowledge about their capabilities and limitations in order to select the right approach for their own purposes, and use it effectively and efficiently.

1.3 Using Formal Methods

The use of formal methods can achieve various goals:

- Obtaining a common and formal description of a system, at the various stages of its development. Formal *specification* allows a common and agreed upon description of the system or its properties that is used by the different groups involved in the development. This description has a clear and unambiguous meaning. It gives the designers, the programmers, the quality assurance teams, and later the users, a collection of documents that can be used to resolve misunderstandings and disputes regarding the intended behavior of the system.
- Formal methods were developed for the purpose of finding errors that are introduced during the development of systems. They are used to help the designer and the developer detect, pinpoint and analyze the errors. Thus, they increase the reliability of the system.
- Formal methods can be integrated with, and assist in, the development process. Some software reliability tools not only allow modeling and capturing the design of a system, but also provide some means of automatically or semiautomatically converting the design into an initial implementation. For example, tools that capture the dynamics of the behavior of a system may attempt to automatically generate code that will exhibit the desired behavior.

Formal methods can be used in almost every stage of a software development project. One can use formal specification to capture the desired features and the system specification. Testing and verification can already start at this early stage. The initial design, be it even a collection of possible scenarios

within a feasibility study, can already be subjected to early fault-detection algorithms. These can alert the developer to some potential problems, such as unexpected interactions between features (feature interaction), or with the environment.

Testing and verification can be applied in different stages of the development. The earlier in the process an error is caught, the less damage it has done, and the cheaper it is to fix it. Yet, even if formal methods were extensively used in early development stages, new errors can still penetrate into the developed system. The repeated use of formal methods in later stages is thus a useful practice.

Software development *methodologies* can prescribe how to compose the system's requirements, plan the timetable, estimate the costs and human resources, divide the work between the different groups, etc. Formal methods are most effectively used within a project when properly used under such a methodology. Specific time and human resources should be reserved for enhancing the reliability of the developed software.

In reality it is still frequently the case that the advocates of formal methods need to compete with other development activities. There are several factors contributing to this situation:

- Formal methods are relatively new. Some of the methods were suggested only recently (twenty years ago, or even less).
- There is an ever-growing inclination to finish the development and ship the products as fast as possible, with tight deadlines on different development activities. On face of it, it seems that formal methods require additional investment in time, money and human resources. The fact that formal methods may save time and money by finding errors is often discounted, in particular in organizations that have not yet had experience with this technology.
- Research in formal methods is frequently focused on improving the expressiveness and efficiency of the techniques. This has often resulted in the neglect of human interface issues, which are crucial in attracting new users. Since formal methods are based on the mathematical analysis of systems, some techniques require the potential users to educate themselves in using a new mathematical framework and notation. It has only recently become widely accepted that the developers of formal methods need to adopt the framework and notation already used by the potential users.

Many software developers and software engineers still have some qualms about the use of formal methods. Perhaps one of the main reasons for this is the use of words such as 'verification,' which suggests some absolute guarantees about the inspected software. Such absolute guarantees cannot be achieved. In particular, a license for a software verification tool typically proclaims no responsibility whatsoever for the detection of some or all of the problems in the inspected software. This seems to be a marketing problem;

it may be solved by better explaining the benefits of using formal methods *and* their limitations.

There are various common misconceptions and prejudices about using formal methods:

Formal methods can be used only by mathematicians.

The fact that formal methods are based on mathematics means that the developers of the methods are likely to be mathematicians, computer scientists or electrical engineers. Indeed, some of the methods, such as deductive verification, require a high level of mathematical skill. However, many of the modern formal methods are designed so that a software engineer can use them with minimal training. This is due to the fact that developers of formal methods have realized that they are often successful when they base their methods on the formalisms and tools of their intended users, rather than requesting that the users learn new formalisms.

Using formal methods will slow down projects.

It is true that using formal methods takes a nonnegligible amount of time. Adding the use of formal methods to the development process requires scheduling a certain number of people to be engaged in this activity for some amount of time. However, experience shows that the use of formal methods, in particular in the early development stages, reduces considerably the time it takes to debug and deliver a system. Verifying or testing a system is time well spent, since customers are frequently willing to pay for reliability.

The verification process itself is prone to errors, so why bother at all?

Indeed, one cannot achieve absolute certainty with regard to the correctness of software. Mistakes can be made during the modeling of software and the translation done in order to fit verification and testing tools. Formal methods tools may also contain bugs, and human verification is obviously prone to errors. In fact, these problems are not merely theoretical and do occur in practice when using formal methods. The important point to realize is that formal methods are not intended to provide *absolute* reliability. The strengths of formal methods are thus in *increasing* the reliability. Statistical studies show that they indeed succeed in achieving this goal.

1.4 Applying Formal Methods

Formal methods can be used in various stages of the software development process, from early design to the acceptance tests of the completed product. Optimally, the use of such methods is integrated into the development process through a methodology. The development timetable should include specific time slots for performing testing and verification tasks. Certifying that the

software has passed certain validity checks can be used as milestones for the development process.

Unfortunately, the effectiveness of formal methods tends to diminish with the size of the checked object. This problem, sometimes called the *state space explosion*, will be discussed at length in this book. Therefore, it is more effective to apply formal methods in the early design and development stages, where the checked objects are relatively small. Another reason for the early use of formal methods is that errors that are found in early stages have less chance of causing a lot of damage and are usually cheaper to fix.

The size of a software system consists nowadays of thousands to millions of lines of code. To cope with the deadlines for software development, software development is often shared among different groups of programmers, sometimes located in different places. Thus, it is often difficult, if not impossible, to apply formal methods to a complete system. Therefore, the tendency is to look for *compositional* methods, which attempt to verify or test different parts of the code separately, and then make conclusions about the system as a whole.

Consider, for example, an idealized case, where a system consists of two subsystems, A and B . We would like to check that the system satisfies some property p . During a compositional verification attempt, we may try to show that A satisfies some property p_1 , and B satisfies some property p_2 . Then, we want to show that given that p_1 and p_2 hold for A and B , respectively, implies that p holds for the complete system. Sometimes, when the development of subsystems A and B is done independently, there is one team that checks A against p_1 , and another team that checks B against p_2 . This allows various interesting possibilities, such as allowing the team that developed A to check B , while the team that developed B checks A (letting the developers check their own code is usually not a good idea, as will be explained in Chapter 9). On the other hand, compositional verification and testing allows increased flexibility in setting the system development schedule. Compositional formal methods are thus very desirable. Unfortunately, as we will see later, there are some difficult cases, where achieving compositionality is hard.

The use of formal methods is not restricted to the code of software systems. One may start by checking the requirements for possible contradictions. The early design, even when it consists merely of a few examples, can also be verified for consistency. Some development methodologies, such as *clean-room* [98, 99] use formal methods throughout the process to refine the code in a way that attempts to preserve its correctness from one development stage to another. Ideally, one can start from the system specification, and by repeatedly refining it, while preserving the correctness at each stage, obtain code that satisfies the specification.

Some of the methods presented in this book are also used for specifying, verifying and testing hardware. In particular, some variants of these methods include optimizations that are most effective for hardware [24]. With the

recent progress in software and hardware design, systems that include both hardware and software components are being developed. Critical software is sometimes implemented on a chip, and hardware design is nowadays done using programming languages. With the blurring of the distinction between software and hardware, and the growing trend of developing systems that include both hardware and software components, we need to provide efficient formal methods techniques that will be able to fit both [80].

Applying formal methods costs money. There is no way around this, although more efficient methods may help in reducing the costs. On the other hand, using formal methods may prevent some unfortunate situations caused by errors, and thus overall, may save money, or in some extreme cases, human life. It is thus not unreasonable to find out that in some cases the investment in applying formal methods may be equivalent or even greater than the investment in all other development activities. Achieving reliability is quite often a good investment, and sometimes is priceless.

1.5 Overview of the Book

Research in formal methods has produced many interesting and useful results. The basic methods demonstrated the capabilities of software analysis. Current improvements are aimed at achieving more efficiency and automatization, in order to apply these methods to a wider range of systems. As the area of formal methods matures, we witness many versions of the basic techniques, some of which are optimized towards more specific applications. It is not possible to cover in an introductory book all the recent results. However, the principles presented in this book are common to many of the basic and current methods.

The two main components of formal methods that are emphasized in this book are *logic* and *automata theory*. With the understanding of these concepts and their use in the formal analysis of systems, it is easy to grasp the basic ideas, as well as the new developments in this area.

We start with some preliminary background from mathematics (Chapter 2). Logic (Chapter 3) is a concisely defined syntax, equipped with a formal semantics and a deductive capability of inferring new facts from given facts. It can be used for formalizing systems and their properties. Certain instances, such as first order logic (Chapter 3) or linear temporal logic (Chapter 5), are frequently used for software specification. Related concepts, such as process algebra (Chapter 8), are used to analyze the subtle issues of interaction within a system or of the system with its environment. Using logical notation is not limited to providing a precise and unambiguous specification. Logics are often equipped with proof systems, which allow one to make inferences manually, or automatically. Such proof systems can be used for program verification (Chapter 7).

Another mathematical theory that plays an important role in formal methods is automata theory (Chapters 5, 6). An automaton is a mathematical structure that can be used to model different behaviors based on the states of the system and the transitions between them. Different versions of automata are often used to model software systems. Automata theory is one of the cornerstones of computer science, and thus many mathematical tools are available for the analysis of automata. Such tools are used in formal methods in order to perform the automatic analysis of systems.

The use of formal methods usually starts with modeling the checked system as a mathematical object (Chapter 4). Formal specification of the modeled system and its required properties (Chapter 5) are given using some selected formalism. Then, one can apply some techniques for obtaining a higher-level of confidence in the correctness of the system. The main methods are model checking (Chapter 6) deductive verification (Chapter 7), and testing (Chapter 9). One can also enhance reliability by stepwise refinement of a design, while maintaining the appropriate conformance between the versions (Chapter 8).

The last chapters of the book describe some more advanced techniques. We show how to combine the strength of different formal methods (Chapter 10). A recent successful trend in formal methods is to use visual formalisms (Chapter 11). We conclude with a discussion of the methods presented, and some new directions in formal methods research (Chapter 12).